

SESSION 5

Django Admin Customization

Topics Covered: Enabling admin, Adding models to admin, Custom list display, Search fields, filters

Activities: Customize admin for Doctor model, Include filters for specialization

Task 1: Enable Django admin interface in your project and create a superuser account to log in to the admin dashboard.

Explanation & Approach: *Superusers possess full authorization privileges to manage all models and user permissions via the Admin UI.*

```
# Step 1: Create admin superuser account
python manage.py createsuperuser

# Terminal Prompts:
# Username: admin
# Email address: admin@foodiehub.com
# Password: *****
# Password (again): *****
# Superuser created successfully!

# Step 2: Access admin at http://127.0.0.1:8000/admin/
```

Task 2: Create Restaurant model with fields name, cuisine, rating and register it with Django admin.

Explanation & Approach: *Registering models with `admin.site.register` exposes them in the Django admin control panel.*

```
# foodiespot/admin.py
from django.contrib import admin
from .models import Restaurant, Cuisine

admin.site.register(Cuisine)
admin.site.register(Restaurant)
```

Task 3: Customize Restaurant admin displaying name, cuisine, and rating in list view using `list_display` option.

Explanation & Approach: *ModelAdmin classes override default single-column string representations in admin lists with tabular columns.*

```
# foodiespot/admin.py
from django.contrib import admin
from .models import Restaurant, Cuisine

@admin.register(Restaurant)
```

```
class RestaurantAdmin(admin.ModelAdmin):
    list_display = ('name', 'cuisine', 'rating', 'location')
```

Task 4: Add search functionality in Restaurant admin so you can search restaurants by name or cuisine using `search_fields` option.

Explanation & Approach: `search_fields` adds an intuitive search box above the admin tabular list view.

```
# foodiespot/admin.py
from django.contrib import admin
from .models import Restaurant

@admin.register(Restaurant)
class RestaurantAdmin(admin.ModelAdmin):
    list_display = ('name', 'cuisine', 'rating', 'location')
    search_fields = ('name', 'cuisine__name', 'location')
```

Task 5: Add a filter in Restaurant admin panel to filter restaurants by cuisine using `list_filter` option.

Explanation & Approach: `list_filter` adds a right sidebar filter pane to filter querysets by categorical relationships.

```
# foodiespot/admin.py (Complete Customization)
from django.contrib import admin
from .models import Restaurant, Cuisine

@admin.register(Restaurant)
class RestaurantAdmin(admin.ModelAdmin):
    list_display = ('name', 'cuisine', 'rating', 'location')
    search_fields = ('name', 'cuisine__name', 'location')
    list_filter = ('cuisine', 'rating')
    list_per_page = 10

@admin.register(Cuisine)
class CuisineAdmin(admin.ModelAdmin):
    list_display = ('name', 'description')
```